

Music / Playlist Organiser

Project Introduction

A menu-driven program that stores songs and playlists in a MySQL database and lets you add, view, search, update and delete them from a Python console application.

9

menu options

3

linked tables

30

sample songs

77/77

automated checks
passed

Contents

1. Overview
2. Objectives
3. Features
4. How it works
5. Database design
6. Project files
7. Requirements
8. Getting started
9. Future scope

Source code: github.com/vjxeyy/music-playlist-organizer

1. Overview

Most people now keep a large digital music collection, and managing it by hand gets difficult: finding a track, grouping songs by genre, or building playlists all take effort. This project solves that with a small database application.

Python is the front end. It shows a menu, reads what the user types, checks it, and displays results as neat tables. **MySQL** is the back end. It stores every song and playlist permanently, so the data is still there the next time the program runs. The two are connected with the `mysql-connector-python` library.

Underneath, the project shows the core ideas of database programming: the four **CRUD** operations (Create, Read, Update, Delete), **primary and foreign keys**, and a **many-to-many relationship** between songs and playlists.

2. Objectives

- Build a useful program that organises songs and playlists through a simple menu.
- Connect Python to MySQL and run SQL queries from Python code.
- Perform all four CRUD operations on real data.
- Design related tables using primary keys, foreign keys and a linking table.
- Handle mistakes gracefully: invalid input, missing records and connection problems.
- Write modular code, with one function per feature.

3. Features

#	Menu option	What it does	Main SQL
1	Add Song	Asks for title, artist, album, genre and duration; warns about duplicates.	<code>INSERT INTO Songs</code>
2	View Songs	Shows every song in a table, with a total count.	<code>SELECT ... ORDER BY song_id</code>
3	Search Song	Finds songs whose title, artist or genre contains a keyword.	<code>SELECT ... WHERE ... LIKE</code>
4	Update Song	Shows the current details; press Enter to keep a value.	<code>UPDATE Songs</code>
5	Delete Song	Shows the song, asks to confirm, and removes it from its playlists.	<code>DELETE FROM Songs</code>
6	Create Playlist	Creates a playlist; names must be unique.	<code>INSERT INTO Playlists</code>
7	Add Song to Playlist	Lists the playlists, then links a song to the chosen one.	<code>INSERT INTO PlaylistSongs</code>
8	View Playlist	Shows a playlist's songs with the song count and total time.	<code>SELECT ... JOIN PlaylistSongs</code>
9	Exit	Ends the program.	–

Beyond the menu itself, the program also:

- **Checks input:** title and artist are required, text must fit its column, and a duration must look like `3:45`.

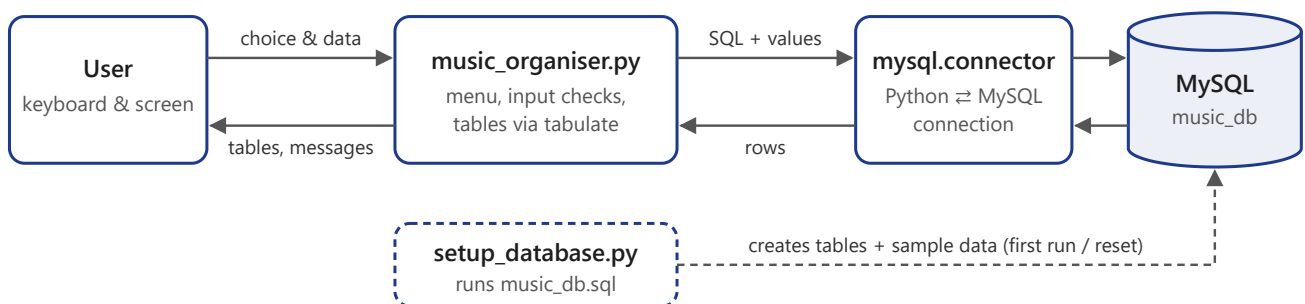
- **Prevents duplicates:** it warns before adding a song that is already stored, and refuses duplicate playlist names or the same song twice in a playlist.
- **Explains problems:** a wrong password, a stopped MySQL server or a missing table produce a clear message instead of a crash.
- **Sets itself up:** on the first run it offers to create the database with sample data.

A real example: adding a song (text typed by the user is **highlighted**).

```
Enter your choice: 1
Enter Song Title: Is There Someone Else?
Enter Artist: The Weeknd
Enter Album: Dawn FM
Enter Genre: R&B
Enter Duration (mm:ss): 3:19

✅ Song added successfully! (Song ID: 31)
```

4. How it works



1. The program shows the menu and the user picks an option, for example 3. *Search Song*.
2. Python reads and checks the input, then builds an SQL query. The user's values are passed separately as parameters (`%s`), which keeps the query safe from SQL injection.
3. `mysql.connector` sends the query to the MySQL server, which runs it on `music_db`.
4. MySQL returns the matching rows. Python formats them as a table with `tabulate` and prints them, then shows the menu again.

Each feature opens a connection, does its work, saves changes with `commit()`, and closes the connection, even if something goes wrong part-way.

5. Database design

The database `music_db` has three tables. A song can be in many playlists, and a playlist holds many songs, so a third table, **PlaylistSongs**, links them.

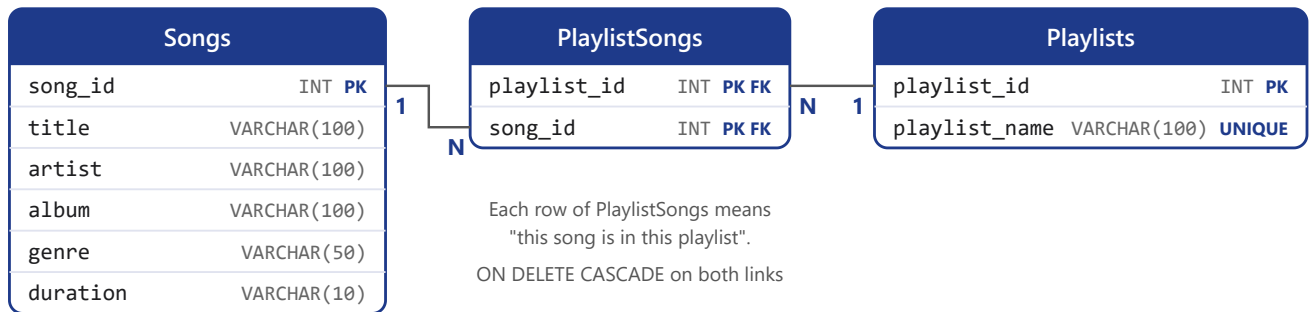


Table	Purpose	Keys
Songs	One row per song: title, artist, album, genre and duration (stored as text like <code>3:20</code>).	<code>song_id</code> primary key, auto-numbered
Playlists	One row per playlist.	<code>playlist_id</code> primary key; <code>playlist_name</code> unique
PlaylistSongs	One row per song-in-playlist link.	Primary key (<code>playlist_id</code> , <code>song_id</code>); both are foreign keys

Why ON DELETE CASCADE? When a song is deleted, MySQL automatically removes its rows from PlaylistSongs. Without it, MySQL would refuse to delete any song that is in a playlist.

6. Project files

File	Purpose
<code>music_organiser.py</code>	The main program: the menu and all nine features.
<code>setup_database.py</code>	Creates or resets the database by running <code>music_db.sql</code> .
<code>db_config.example.py</code>	Template for <code>db_config.py</code> , which holds the MySQL username and password.
<code>music_db.sql</code>	Creates the three tables and loads 30 sample songs and 6 playlists.
<code>sample_queries.sql</code>	Twelve example SQL queries on the sample data.
<code>requirements.txt</code>	The Python libraries to install.
<code>docs/working-output.pdf</code>	Real captured output of every feature.

7. Requirements

Software	Version
Python	3.8 or later
MySQL Server	8.0 or later
Python libraries	<code>mysql-connector-python</code> , <code>tabulate</code>
Editor (any)	IDLE, VS Code or PyCharm

Tested on Windows 11 with Python 3.14.5, MySQL Server 26.7.0, `mysql-connector-python` 26.7.0 and `tabulate` 0.10.0.

8. Getting started

1. Install MySQL Server and make sure it is running.
2. Install the libraries: `pip install -r requirements.txt`
3. Copy `db_config.example.py` to `db_config.py` and put your MySQL password in it.
4. Run `python music_organiser.py`. On the first run, answer **y** to create the database with the sample data.
5. To restore the sample data later, run `python setup_database.py`.

9. Future scope

- **User accounts:** log in so each person has their own songs and playlists.
- **Playlist sharing:** let users share playlists with each other.
- **Graphical interface:** windows and buttons (for example with Tkinter) instead of a text menu.
- **More playlist tools:** remove a song from a playlist, rename or delete a playlist.
- **Duration in seconds:** store duration as a number, so sorting and comparing work for songs of any length.

See `docs/working-output.pdf` for the real output of every feature, and github.com/vjxeyy/music-playlist-organizer for the source code.